

C++ · SVNIT SURAT · THE TU CHAHIYE QUIZ, ON PAPER

OBJECT ORIENTED PROGRAMMING

40 QUESTIONS

90 MARKS

45 MINUTES

ANSWER KEY AT THE BACK

How the marks work. Ten easy questions at 1 mark, ten medium at 2, and twenty hard at 3 — ninety in total, two thirds of them in the hard twenty. One option is right in every question. **The answer key, with a short explanation for every question, starts on the last section of this document.**

40

QUESTIONS

90

MARKS

45:00

ON THE CLOCK

1

ATTEMPT ONLINE

01

Who created the C++ language?

1 MARK

A Dennis Ritchie

B Bjarne Stroustrup

C James Gosling

D Guido van Rossum

02

How many keywords does C++ have?

1 MARK

A 32

B 48

C 63

D 72

03

Which operator is known as the extraction or “get from” operator?

1 MARK

A <<

B >>

C ::

D ->

04

What is the return type of a constructor?

1 MARK

A void

B int

C The same type as the class

D It has no return type at all

05

A destructor's name is the name of the class preceded by which symbol?

1 MARK

A !

B ~

C #

D &

06

In a class, members written before any access specifier are:

1 MARK

A public

B private

C protected

D static

- 07** Which operator did C++ add so that a global variable hidden by a local one of the same name can still be reached? 1 MARK
- A `.*` B `->`
C `::` D `&`
- 08** Which pair of operators does C++ provide for dynamic memory management? 1 MARK
- A `malloc` and `free` B `new` and `delete`
C `alloc` and `dealloc` D `create` and `destroy`
- 09** In object oriented programming, the emphasis is on: 1 MARK
- A The algorithm — the steps to be performed B Data rather than procedure
C Sharing global variables efficiently D The top-down approach to design
- 10** Which header must be included to use `setw()` and `setprecision()`? 1 MARK
- A `<iostream>` B `<cmath>`
C `<iomanip>` D `<csdarg>`
- 11** How much memory does a union occupy compared to a structure with the same members? 2 MARKS
- A Equal to the sum of the sizes of all its members B Equal to the size of its largest member
C Always exactly one byte D Equal to the size of its smallest member
- 12** Given `enum color {red = 5, blue, green};` what is the value of `green`? 2 MARKS
- A 2 B 5
C 6 D 7
- 13** Which storage class gives a variable the visibility of a local variable but the lifetime of an external variable? 2 MARKS
- A `auto` B `register`
C `extern` D `static`
- 14** In C++, what size must the array be in `char s[] = "xyz";`? 2 MARKS
- A 3 B 4
C 5 D Any size is accepted
- 15** Default values for function arguments: 2 MARKS
- A May be given to any argument you like B May be given only to trailing arguments
C Must be given to every argument or none D Must appear in the function definition, not the prototype

- 23** An object `B` has `operator+` overloaded as a MEMBER function. What happens with `A = 2 + B;`? 3 MARKS
- A It works — the compiler converts 2 into an object first
- B It fails, because it becomes `2.operator+(B)` and an `int` has no member functions
- C It works only if `B` is declared `const`
- D It fails, because `operator+` cannot accept an `int` at all
- 24** Objects `A1`, `A2` and `A3` are created in that order inside a block. In what order are their destructors called? 3 MARKS
- A `A1`, then `A2`, then `A3`
- B `A3`, then `A2`, then `A1`
- C The order is undefined
- D All three at exactly the same moment
- 25** A class declares `static int count;` inside it, but no definition appears anywhere outside the class. What happens? 3 MARKS
- A It compiles and `count` starts at 0
- B A compile error is reported at the declaration line
- C It compiles, but fails at link time with “undefined reference”
- D Each object silently gets its own private count
- 26** Why is `int f(char*); long int f(char*);` not valid function overloading? 3 MARKS
- A Because `char*` may never be used as an overload parameter
- B Because the two functions differ only in return type, and a call does not reveal which return type was wanted
- C Because `long int` and `int` are treated as the same type in C++
- D It is valid — the compiler picks by the type of the variable being assigned
- 27** What is the result of `typedef int INT;` followed by `void display(int);` and `void display(INT);`? 3 MARKS
- A Two perfectly valid overloads
- B An error, because `typedef` only renames an existing type and does not create a new one
- C The `INT` version is silently ignored by the compiler
- D It compiles in C but not in C++
- 28** A class has both `complex()` and `complex(float r = 0, float i = 0)`. What is wrong with `complex c;`? 3 MARKS
- A Nothing — the first constructor is chosen
- B Nothing — the second constructor is chosen, using both defaults
- C It is ambiguous, because both constructors can be called with zero arguments
- D An error, because a class may not have two constructors
- 29** Overloaded as a FRIEND function, a unary operator takes how many arguments? 3 MARKS
- A Zero
- B One
- C Two
- D Three

30 Which operator can be overloaded, but ONLY as a member function and never as a friend? 3 MARKS

A + B <<

C = D ==

31 What does this program print? 3 MARKS

```
class item {
    static int count;
    int number;
public:
    void get() { count++; }
    void show() { cout << count << " "; }
};
```

```
int item::count;
```

```
int main() {
    item a, b, c;
    a.get();
    b.get();
    a.show(); b.show(); c.show();
    return 0;
}
```

A 1 1 0 **B** 2 2 2
C 1 2 0 **D** 0 0 0

32 Given `int (Data::*fp)(float) = &Data::f;` and an object `d`, which call is correct? 3 MARKS

A `d.*fp(3.5);` B `(d.*fp)(3.5);`

C `d->*fp(3.5);` D `fp(d, 3.5);`

33 A class allocates `char *p` with `new` in its constructor and frees it with `delete[]` in its destructor, but has NO copy constructor. An object of it is passed to a function by value. What happens?

A Nothing unusual — the object behaves correctly B The string is safely duplicated by the compiler

C The pointer itself is copied, so two objects point at the same memory, and when both destructors run that memory is deleted twice — the program crashes D The compiler refuses to compile the function call

34 A class defines ONLY the constructor `X(int a)`. What happens with the statement `X obj;`? 3 MARKS

A It compiles and `a` holds garbage B It compiles and `a` is set to 0

C A compile error, because writing any constructor removes the compiler-supplied default one D It quietly calls the copy constructor instead

- 35** Which statement about friend functions is TRUE? 3 MARKS
- A Friendship is mutual — if A declares B a friend, B is automatically a friend of A
- B Friendship is inherited, so a friend of a base class is a friend of its derived classes
- C A friend function is called like an ordinary function, without the dot operator
- D A friend function can name private members directly, without going through an object
-
- 36** In this program, which destructor runs FIRST? 3 MARKS
- ```
int main()
{
 α A1, A2, A3;
 {
 α A4;
 }
 cout << "back in main";
 return 0;
}
```
- A A1
- B A4
- C A3
- D All four run together at the end of main
- 
- 37** Consider `X b = a;` on one line, and then `b = a;` on a later line. Which function runs in each case? 3 MARKS
- A Both call the copy constructor
- B Both call the assignment operator
- C The first calls the copy constructor, the second calls the assignment operator
- D The first calls the assignment operator, the second calls the copy constructor
- 
- 38** Inside a member function of class `complex`, the line `temp.x = x + c.x;` reads `c.x` — a private member of a DIFFERENT object. Is this legal? 3 MARKS
- A No — private means private to each individual object
- B Yes — access control in C++ is per class, not per object
- C Only if `c` was passed by reference rather than by value
- D Only if `complex` declares itself a friend of itself
- 
- 39** Why is C++ far stricter about type compatibility than C — for example treating `char`, `signed char` and `unsigned char` as three distinct types? 3 MARKS
- A To make compiled programs run measurably faster
- B To reduce the memory a program uses
- C To support function overloading, which needs genuinely distinct types in order to tell one overload from another
- D Because the `new` and `delete` operators require it
- 
- 40** Which statement about `new/delete` compared with `malloc()/free()` is TRUE? 3 MARKS
- A `new` and `delete` are library functions declared in `<stdlib.h>`
- B `new` returns `void*` and its result usually has to be cast
- C `new` calls the constructor and `delete` calls the destructor; `malloc` and `free` do neither
- D `new` and `delete` cannot be used with user-defined types such as classes

# The answer key

The letter, then why. Read the explanation even where you got it right — that is where most of the revision value is.

- 01     **B**     C was created by Brian Kernighan and Dennis Ritchie. C++ was created by Bjarne Stroustrup, and it started life as “C with Classes” — keep everything good about C, add the ability to bundle data with the functions that guard it.
- 02     **C**     C++ has 63 keywords. A keyword (or reserved word) has a meaning fixed by the language and can never be used as the name of your own variable or function. All of them are in lower case.
- 03     **B**     `cin >> x;` extracts a value from the keyboard and assigns it to the variable on its right. Its partner `<<` is the insertion or “put to” operator, used with `cout`.
- 04     **D**     A constructor has no return type — not even `void`. Writing one is a compile error. The same is true of a destructor. A constructor also has the same name as its class, is called automatically, and should be declared `public`.
- 05     **B**     The tilde. For a class `integer` the destructor is `~integer()`. It takes no arguments, returns no value, and there can be only one per class — a destructor cannot be overloaded.
- 06     **B**     Members of a `class` are private by default. This is the one real difference between `class` and `struct` in C++ — members of a `struct` are public by default.
- 07     **C**     The scope resolution operator. `::m` means the global version of `m`, not the local one declared in the current block. In C this was impossible — a hidden global could not be reached at all from the inner block.
- 08     **B**     `new` and `delete` are unary operators built into the language. `malloc()` and `free()` are C library functions — they still work in C++, but they do not call constructors or destructors.
- 09     **B**     OOP puts data first and does not allow it to flow freely around the system. Procedure oriented programming is the opposite — its emphasis is on doing things, and it uses a top-down approach with shared global data.
- 10     **C**     `setw`, `setfill` and `setprecision` all live in `<iomanip>`. Only `endl` comes free with `<iostream>`. Forgetting `<iomanip>` gives a confusing “was not declared in this scope” error.
- 11     **B**     In a union all members share one piece of memory, so its size is that of its largest member. A structure gives each member its own storage, so its size is at least the sum of them. That is also why altering one member of a union alters the others, while a structure’s members are independent.
- 12     **D**     When you assign a value to one enumerator, the ones after it continue counting up from there. So `red` is 5, `blue` is 6 and `green` is 7. Without any explicit value the list would have started at 0.
- 13     **D**     A `static` variable declared inside a function is not destroyed when the function ends — it retains its value for the next call — but it is still only visible inside that function. It also starts at zero, whereas `auto` and `register` start as garbage.
- 14     **B**     C++ requires room for the terminating null character `\0`, so the size must be one larger than the number of characters — 4. ANSI C is more relaxed and allows `char s[3] = "xyz";`, assuming you meant to leave the null out.
- 15     **B**     Only trailing arguments may have defaults, because arguments are matched left to right — with a gap in the middle the compiler could not tell which argument a value was meant for. The defaults also go in the prototype, not the definition.
- 16     **D**     Exactly four operators can never be overloaded: `::`, `.`, `*` and `?:`. The subscript operator `[]` can be overloaded — though only as a member function, never as a friend.
- 17     **B**     Member functions are placed in memory once, when the class is defined, and all objects share that single copy. Member variables are the opposite — space for them is allocated separately for each object, because they hold different values in different objects.
- 18     **C**     `do...while` tests its condition at the bottom, so the body always runs at least once. `while` and `for` test at the top and may run zero times. Note also the semicolon that `do...while` requires after its closing condition.

- 19 **B** `int &r;` on its own is an error — a reference must be bound when it is declared. And once bound it can never be re-pointed at a different variable: a later `r = y;` copies `y`'s value into whatever `r` already refers to.
- 20 **B** A generic pointer may be assigned a pointer of any basic type, but it may never be dereferenced. `*gp` is meaningless — the compiler has no idea how many bytes to read or how to interpret them.
- 21 **A** `k` is initialised before the inner `m` is declared, so it picks up `main`'s `m` — 20. Then the innermost declaration wins, so `m` prints 30. And `:m` always jumps straight to the global one, 10, no matter how deep you are.
- 22 **B** Passing an object by value means making a copy, and making a copy means calling the copy constructor. So `A(A)` would call itself to construct its own argument, forever. Taking the argument by reference breaks the loop, which is why the copy constructor must use `&`.
- 23 **B** For a member operator function, `a + b` becomes `a.operator+(b)` — the left operand becomes the calling object. 2 is an `int`, and an `int` has no member functions. Written as a friend it becomes `operator+(2, B)`, an ordinary two-argument call, and both orders work. This is the main reason friend functions exist.
- 24 **B** Objects are destroyed in the reverse order of their creation, at the closing brace of the block they were declared in. If a fourth object were created inside a nested block, it would be destroyed first — at the end of that inner block, before the outer ones.
- 25 **C** Inside the class you only declare that a static member exists — no storage is allocated. The line `int item::count;` outside the class is what actually allocates it. Leave it out and the compiler is satisfied but the linker is not.
- 26 **B** Overloaded functions must differ in the type, number or order of their parameters. Imagine the call written on its own line with the result discarded: the compiler sees only the arguments, which are identical, so it cannot choose. Difference in return type alone is never enough.
- 27 **B** You cannot fool the compiler by renaming a type. `INT` is just another name for `int`, so the two declarations are the same function and the compiler cannot tell them apart. Contrast this with `char*` and `const char*`, which genuinely are different types and can be overloaded.
- 28 **C** Both are valid candidates for a call with no arguments, so the compiler cannot choose and reports an ambiguity. Having a default constructor and an all-defaults parameterized constructor in the same class is always a mistake — write one or the other.
- 29 **B** A friend function always needs one more argument than the member version, because it has no calling object to supply the left operand. So: unary as member 0, unary as friend 1; binary as member 1, binary as friend 2.
- 30 **C** Four operators must be members: `=` `()` `[]` and `->`. Do not confuse this list with the four that cannot be overloaded at all — `::` `.*` and `?:`. Two different lists, and both are commonly examined.
- 31 **B** There is only one `count` for the whole class, shared by every object. `a.get()` makes it 1 and `b.get()` makes it 2. All three objects then read that same shared copy, so every `show()` prints 2 — including `c`, which never called `get()` at all.
- 32 **B** The outer brackets are compulsory. C++ gives the function-call brackets higher precedence than `.*`, so without them `d.*fp(3.5)` is read as `d.*(fp(3.5))` — it tries to call `fp` first, which is a guaranteed compile error.
- 33 **C** Passing by value makes a copy, and the compiler's default copy just duplicates the pointer, not the memory it points at. Both objects now own the same block, and both destructors free it. This is exactly why any class that allocates memory with `new` needs its own copy constructor and destructor.
- 34 **C** C++ supplies a free default constructor only when you write none of your own. The moment you declare any constructor, that free one disappears. If you want both `X obj;` and `X obj(5);` to work, you must write both constructors yourself.
- 35 **C** A friend is not a member, so it is called as `mean(X)`, never `X.mean()`. The other three are all false: friendship is granted not taken, it is not mutual, it is not inherited, and inside the function you must write `s.a` — it has no object of its own to work from.

- 36     **B**     Destruction follows scope exactly. A4 was declared in the inner block, so it is destroyed at that block's closing brace — before “back in main” is even printed. A3, A2 and A1 then follow, in that order, when main ends.
- 37     **C**     The deciding question is whether the object already exists. `X b = a;` is creating b, so it is copy initialization and the copy constructor runs. By the later line b already exists, so `b = a;` is plain assignment.
- 38     **B**     Privacy in C++ is enforced per class. A member function of `complex` may reach the private members of any `complex` object, not only the one it was called on. Without this rule you could never write a function that adds, compares or copies two objects of the same class.
- 39     **C**     If `char` and `signed char` were the same type, the compiler could not tell `display(char)` from `display(signed char)` and overloading would collapse. Strict types are the price C++ pays for letting one name mean several things.
- 40     **C**     This is the difference that matters in an OOP course. `new` builds a real object — it runs the constructor — and `delete` runs the destructor before releasing the memory. `malloc` just hands you raw bytes. `new` is also an operator, needs no header, works out the size itself, and returns the correct pointer type with no cast.

C++ · OBJECT ORIENTED PROGRAMMING · PRINTED FROM TU CHAHIYE  
THE LIVE VERSION KEEPS A LEADERBOARD AND ALLOWS ONE ATTEMPT PER NAME